

```

import heapq

class PuzzleState:
    def __init__(self, board, g_cost, goal_positions, parent=None, action=None):
        self.board = board
        self.g_cost = g_cost
        self.parent = parent
        self.action = action # Tracks the direction string ("Up", "Down", etc.)
        self.goal_positions = goal_positions
        self.h_cost = self.calculate_manhattan_distance()
        self.f_cost = self.g_cost + self.h_cost

    def calculate_manhattan_distance(self):
        distance = 0
        for i, val in enumerate(self.board):
            if val != 0:
                current_row, current_col = i // 3, i % 3
                goal_row, goal_col = self.goal_positions[val]
                distance += abs(current_row - goal_row) + abs(current_col - goal_col)
        return distance

    def __lt__(self, other):
        return self.f_cost < other.f_cost

    def get_neighbors(self):
        neighbors = []
        blank_idx = self.board.index(0)
        row, col = blank_idx // 3, blank_idx % 3

        # Directions corresponding to moving the BLANK space
        moves = [(-1, 0, "Up"), (1, 0, "Down"), (0, -1, "Left"), (0, 1, "Right")]

        for dr, dc, move_name in moves:
            new_row, new_col = row + dr, col + dc
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_blank_idx = new_row * 3 + new_col
                new_board = list(self.board)
                new_board[blank_idx], new_board[new_blank_idx] = new_board[new_blank_idx], new_board[blank_idx]
                neighbors.append(PuzzleState(tuple(new_board), self.g_cost + 1, self.goal_positions, self, move_name))

        return neighbors

def print_board(board):
    """Helper function to print the 8-puzzle board in a nice 3x3 layout."""
    for i in range(0, 9, 3):
        row_str = " ".join(str(x) if x != 0 else "_" for x in board[i:i+3])
        print(f"    [ {row_str} ]")

def solve_8_puzzle(initial_board, goal_board):
    goal_positions = {val: (i // 3, i % 3) for i, val in enumerate(goal_board)}
    initial_state = PuzzleState(initial_board, 0, goal_positions)

    open_list = []
    heapq.heappush(open_list, initial_state)
    closed_set = set()

    while open_list:
        current_state = heapq.heappop(open_list)

        if current_state.board == goal_board:
            # Trace backward to collect the path states taken
            path = []
            while current_state is not None:
                path.append(current_state)
                current_state = current_state.parent
            return path[::-1] # Reverse to show chronological order from start to goal

        closed_set.add(current_state.board)

        for neighbor in current_state.get_neighbors():
            if neighbor.board in closed_set:
                continue
            heapq.heappush(open_list, neighbor)

    return None

```

```

def count_inversions(board):
    flat_list = [num for num in board if num != 0]
    inversions = 0
    for i in range(len(flat_list)):
        for j in range(i + 1, len(flat_list)):
            if flat_list[i] > flat_list[j]:
                inversions += 1
    return inversions

def is_solvable(start, goal):
    return count_inversions(start) % 2 == count_inversions(goal) % 2

def get_user_input(prompt_text):
    print(prompt_text)
    print("Enter 9 numbers (0-8) separated by spaces, row by row.")
    print("Example: 2 8 3 1 6 4 7 0 5")
    while True:
        try:
            user_input = input("Input: ").strip().split()
            if len(user_input) != 9:
                print("Error: Please enter exactly 9 numbers.")
                continue
            board = tuple(int(x) for x in user_input)
            if sorted(board) != list(range(9)):
                print("Error: Input must contain numbers from 0 to 8 with no duplicates.")
                continue
            return board
        except ValueError:
            print("Error: Invalid characters detected. Please use numbers only.")

if __name__ == "__main__":
    print("=" * 45)
    print("NAME: HRITHIKA RAJESH")
    print("USN : 1BM24CS120")
    print("=" * 45)
    print("=== 8-PUZZLE A* DETAILED SOLVER ===\n")

    initial_state = get_user_input("--- ENTER INITIAL STATE ---")
    print()
    goal_state = get_user_input("--- ENTER GOAL STATE ---")
    print("\n-----")

    if not is_solvable(initial_state, goal_state):
        print("\n[!] Error: This puzzle configuration is mathematically unsolvable.")
    else:
        print("\nSolving using A* Search (Manhattan Distance)...")
        path_states = solve_8_puzzle(initial_state, goal_state)

        if path_states is not None:
            print(f"\n[+] Success! Optimal path requires {len(path_states) - 1} moves.\n")
            print("=====")
            print("          DETAILED STEP-BY-STEP PATH          ")
            print("=====")

            for index, state in enumerate(path_states):
                if index == 0:
                    print(f"\n👉 STEP 0: INITIAL STATE")
                else:
                    print(f"\n👉 STEP {index}: Move Blank Space '{state.action.upper()}'")

                print_board(state.board)
                print(f"          Metrics -> g(n) = {state.g_cost} | h(n) = {state.h_cost} | f(n) = {state.f_cost}")
                print("-" * 41)

            print(f"\n🎉 GOAL STATE SUCCESSFULLY REACHED!")
        else:
            print("\n[-] Could not find a path to the goal state.")

```

```
[+] Success! Optimal path requires 5 moves.

=====
      DETAILED STEP-BY-STEP PATH
=====

👉 STEP 0: INITIAL STATE
  [ 2 8 3 ]
  [ 1 6 4 ]
  [ 7 _ 5 ]
  Metrics -> g(n) = 0 | h(n) = 5 | f(n) = 5
  -----

👉 STEP 1: Move Blank Space 'UP'
  [ 2 8 3 ]
  [ 1 _ 4 ]
  [ 7 6 5 ]
  Metrics -> g(n) = 1 | h(n) = 4 | f(n) = 5
  -----

👉 STEP 2: Move Blank Space 'UP'
  [ 2 _ 3 ]
  [ 1 8 4 ]
  [ 7 6 5 ]
  Metrics -> g(n) = 2 | h(n) = 3 | f(n) = 5
  -----

👉 STEP 3: Move Blank Space 'LEFT'
  [ _ 2 3 ]
  [ 1 8 4 ]
  [ 7 6 5 ]
  Metrics -> g(n) = 3 | h(n) = 2 | f(n) = 5
  -----

👉 STEP 4: Move Blank Space 'DOWN'
  [ 1 2 3 ]
  [ _ 8 4 ]
  [ 7 6 5 ]
  Metrics -> g(n) = 4 | h(n) = 1 | f(n) = 5
  -----

👉 STEP 5: Move Blank Space 'RIGHT'
  [ 1 2 3 ]
  [ 8 _ 4 ]
  [ 7 6 5 ]
  Metrics -> g(n) = 5 | h(n) = 0 | f(n) = 5
  -----

🎉 GOAL STATE SUCCESSFULLY REACHED!
```